

Dependency Parsing

Phân tích phụ thuộc trong Xử lý ngôn ngữ tự nhiên

Nhóm 4

Môn học: Xử lý ngôn ngữ tự nhiên

Ngày 18 tháng 5 năm 2026

- 1 Giới thiệu
- 2 Dependency Relations
- 3 Dependency Formalism
- 4 Projectivity
- 5 Dependency Treebanks
- 6 Transition-Based Dependency Parsing
- 7 Creating an Oracle

- 1 Giới thiệu
- 2 Dependency Relations
- 3 Dependency Formalism
- 4 Projectivity
- 5 Dependency Treebanks
- 6 Transition-Based Dependency Parsing
- 7 Creating an Oracle

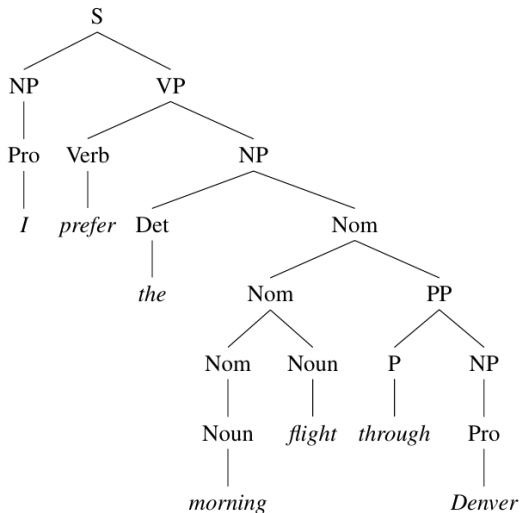
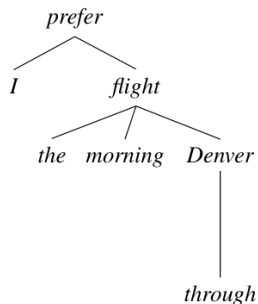
Dependency Grammar

Mô hình biểu diễn cú pháp trong đó cấu trúc câu được xây dựng từ các quan hệ phụ thuộc giữa các từ.

Đặc điểm

- Cú pháp được mô tả bằng các quan hệ có hướng giữa các từ
- Dùng các cạnh nối Head → Dependent

Dependency Parsing vs Constituency Parsing



Dependency vs Constituency Parsing

Dependency Parsing

- Quan hệ giữa từ
- Không có node phrase
- Đơn giản hơn

Constituency Parsing

- Dựa trên phrase structure
- Có NP, VP, PP
- Tree lớn hơn

Ưu điểm lớn của dependency parsing:

- Liên kết trực tiếp predicate và argument
- Phù hợp cho NLP hiện đại

Typed Dependency Structure

- Mỗi dependency arc không chỉ nối:

Head $\rightarrow$ Dependent

mà còn có một **label/type**

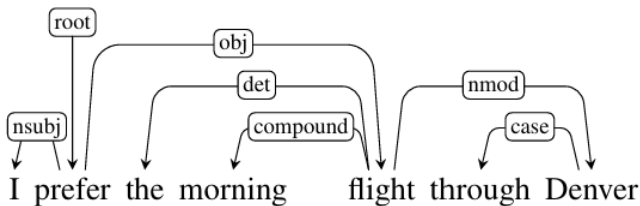
- Label mô tả chức năng ngữ pháp của dependent đối với head
- Các labels được lấy từ một tập dependency relations cố định
- Ví dụ:

prefer $\xrightarrow{\text{obj}}$ flight

Typed Dependency Structure

Sentence:

I prefer the morning flight through Denver



Typed Dependency = Dependency Structure + Relation Labels

- 1 Giới thiệu
- 2 Dependency Relations**
- 3 Dependency Formalism
- 4 Projectivity
- 5 Dependency Treebanks
- 6 Transition-Based Dependency Parsing
- 7 Creating an Oracle

Dependency Relation

Quan hệ ngữ pháp có hướng giữa hai từ:

Head $\rightarrow$ Dependent

Head

- Từ trung tâm
- Quyết định chức năng chính
- Có thể có nhiều dependent

Dependent

- Từ phụ thuộc vào head
- Bổ sung ý nghĩa cho head

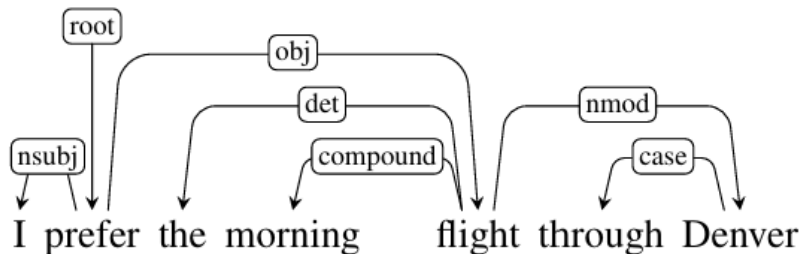
Ví dụ:

prefer $\rightarrow$ flight (obj)

Các Dependency Relations phổ biến

Relation	Ý nghĩa	Ví dụ
NSUBJ	Nominal Subject	United canceled the flight
OBJ	Direct Object	United diverted the flight
IOBJ	Indirect Object	We booked her the flight
COMPOUND	Compound Modifier	morning flight
NMOD	Nominal Modifier	flight to Houston
AMOD	Adjectival Modifier	cheapest flight
DET	Determiner	the flight
CASE	Preposition / Case Marker	through Houston
CC	Coordinating Conjunction	and / or
CONJ	Conjunct Relation	flights and hotels

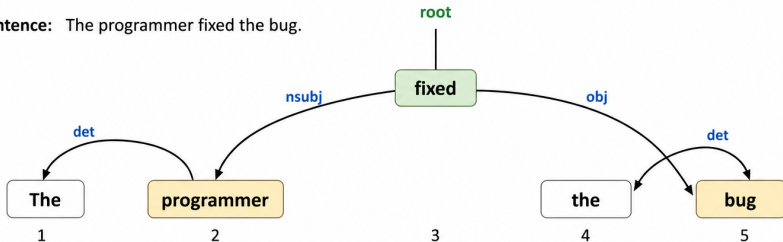
Ví dụ Dependency Relation



The programmer fixed the bug.

Dependency Parsing

Sentence: The programmer fixed the bug.



Word	Index	Head	Relation	Ý nghĩa	Giải thích
The	1	programmer (2)	det	Determiner	Mạo từ "The" xác định danh từ "programmer".
programmer	2	fixed (3)	nsubj	Nominal Subject	"programmer" là chủ ngữ của động từ "fixed".
fixed	3	ROOT	root	Root	Động từ chính của câu.
the	4	bug (5)	det	Determiner	Mạo từ "the" xác định danh từ "bug".
bug	5	fixed (3)	obj	Direct Object	"bug" là đối tượng trực tiếp bị tác động bởi "fixed".

Chú thích các quan hệ

- **root**: gốc của cây (động từ chính)
- **nsubj**: chủ ngữ danh từ
- **obj**: tân ngữ trực tiếp
- **det**: mạo từ

- 1 Giới thiệu
- 2 Dependency Relations
- 3 Dependency Formalism**
- 4 Projectivity
- 5 Dependency Treebanks
- 6 Transition-Based Dependency Parsing
- 7 Creating an Oracle

Dependency Structure

Dependency structure được biểu diễn bằng:

$$G = (V, A)$$

- V : các từ trong câu
- A : các quan hệ phụ thuộc

Head $\longrightarrow$ Dependent

Ví dụ

`fixed` $\rightarrow$ `bug`

Dependency Tree Constraints

Một dependency tree cần:

- 1 Một **root** duy nhất
- 2 Root không có incoming arc
- 3 Mỗi node chỉ có **một head**
- 4 Có đường đi từ root tới mọi node

Kết quả

- Connected tree
- Không cycle
- Quan hệ rõ ràng giữa các từ

- 1 Giới thiệu
- 2 Dependency Relations
- 3 Dependency Formalism
- 4 Projectivity**
- 5 Dependency Treebanks
- 6 Transition-Based Dependency Parsing
- 7 Creating an Oracle

Projective Arc

Một arc từ *head* đến *dependent* được gọi là **projective** nếu mọi từ nằm giữa chúng trong câu:

- đều có thể được truy cập từ head
- thông qua các đường đi trong dependency tree

Projective arc $\Rightarrow$ không tạo cạnh cắt nhau

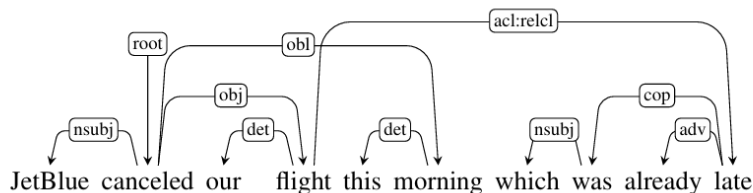
Projective Tree

- Tất cả các arcs đều projective
- Có thể vẽ cây mà không có crossing edges
- Phổ biến trong English dependency treebanks

Non-projective Tree

- Xuất hiện các cạnh giao nhau
- Thường gặp ở ngôn ngữ có word order linh hoạt
- Khó xử lý hơn đối với transition-based parsers

Ví dụ Non-projective Tree



(19.3)

- 1 Giới thiệu
- 2 Dependency Relations
- 3 Dependency Formalism
- 4 Projectivity
- 5 Dependency Treebanks**
- 6 Transition-Based Dependency Parsing
- 7 Creating an Oracle

Dependency Treebank

Tập dữ liệu gồm các câu đã được gán nhãn **dependency trees** nhằm phục vụ huấn luyện và đánh giá parser.

Vai trò của Treebanks

- Huấn luyện dependency parsers
- Cung cấp **nhãn chuẩn (gold labels)** để đánh giá độ chính xác
- Hỗ trợ nghiên cứu ngôn ngữ học ngữ liệu và NLP

Cách xây dựng

- Người gán nhãn trực tiếp annotate dependency trees
- Chỉnh sửa kết quả từ các automatic parsers
- Chuyển đổi từ constituency / phrase-structure treebanks

Universal Dependencies (UD)

Dự án mã nguồn mở xây dựng dependency treebanks theo cùng một chuẩn annotation cho nhiều ngôn ngữ.

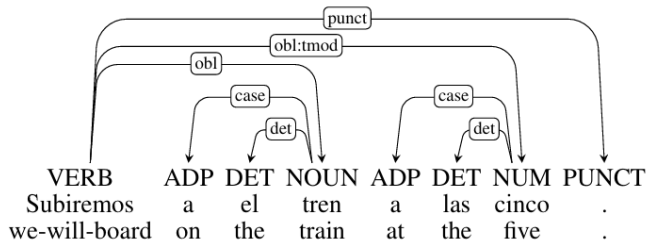
Quy mô hiện tại

- Gần **200 dependency treebanks**
- Hỗ trợ hơn **100 ngôn ngữ**

Ý nghĩa trong NLP

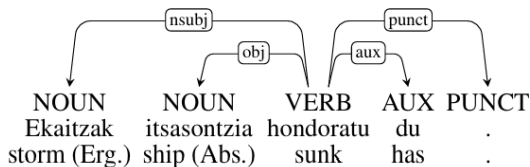
- Giúp các mô hình học cú pháp trên nhiều ngôn ngữ
- Hỗ trợ **cross-lingual NLP**
 - transfer knowledge giữa các ngôn ngữ
- Hỗ trợ **multilingual dependency parsing**

Ví dụ UD: Spanish



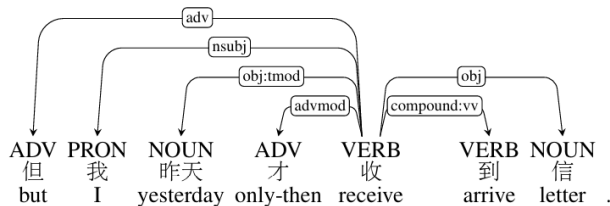
[Spanish] Subiremos al tren a las cinco. “We will be boarding the train at five”(19.4)

Ví dụ UD: Basque



[Basque] Ekaitzak itsasontzia hondoratu du. "The storm has sunk the ship." (19.5)

Ví dụ UD: Chinese



[Chinese] 但我昨天才收到信 “But I didn’t receive the letter until yesterday”(19.6)

Transition-Based Dependency Parsing

Ta cần trả lời 3 câu hỏi chính

- ① **Transition-based dependency parsing** hoạt động như thế nào?
- ② Parser dùng những thao tác nào?
- ③ **Oracle** được tạo từ treebank ra sao để huấn luyện mô hình chọn hành động?

Transition-Based Parsing Approach

Transition-based parsing xây cây phụ thuộc bằng một chuỗi hành động rời rạc.

Stack	Input Buffer	Oracle
Lưu các từ đã đọc nhưng có thể vẫn cần gắn quan hệ phụ thuộc.	Chứa các token chưa xử lý, parser đọc từ trái sang phải.	Bộ dự đoán hành động tiếp theo dựa trên cấu hình hiện tại.

Điểm cốt lõi

Mỗi bước parser chỉ nhìn trạng thái hiện tại rồi chọn một transition để tiến gần hơn tới dependency tree hoàn chỉnh.

Biểu diễn trạng thái: Configuration

Một trạng thái của parser được gọi là **configuration**:

$$C = (S, B, R)$$

Thành phần	Ý nghĩa
S	Stack: chứa các token đã được shift từ buffer sang.
B	Buffer: danh sách các token còn chờ xử lý.
R	Tập các quan hệ phụ thuộc đã được tạo, ví dụ (<i>head</i> $\rightarrow$ <i>dependent</i>).

Biểu diễn trạng thái: Configuration

Trạng thái đầu và trạng thái kết thúc

- Ban đầu: $S = [\text{ROOT}]$, $B = [w_1, w_2, \dots, w_n]$, $R = \emptyset$.
- Kết thúc: mọi từ đã được xử lý, stack chỉ còn $[\text{ROOT}]$, và R là dependency tree cuối cùng.

Ba loại lựa chọn

Khi đọc câu từ trái sang phải, parser thường gặp 3 lựa chọn:

- ➊ **Gắn từ hiện tại làm head** của một từ đã thấy trước đó.
- ➋ **Gắn một từ đã thấy trước đó làm head** của từ hiện tại.
- ➌ **Trì hoãn quyết định**: đưa từ hiện tại vào stack để xử lý sau.

Từ trực giác đến toán tử

Ba lựa chọn trên được chuẩn hóa thành ba transition chính:

LEFT, RIGHT, SHIFT

Arc-Standard Approach to Transition-Based parsing

Hệ thao tác trên là **arc-standard**. Các thao tác tạo quan hệ chỉ xét **hai phần tử trên cùng của stack**.

Transition	Tác dụng
LEFTARC	Gắn từ ở đỉnh stack làm head của từ ở vị trí thứ hai ; sau đó bỏ từ thứ hai khỏi stack.
RIGHTARC	Gắn từ ở vị trí thứ hai làm head của từ ở đỉnh stack ; sau đó bỏ từ ở đỉnh stack.
SHIFT	Lấy token đầu tiên trong buffer và đẩy vào stack.

Nhận xét

LEFTARC và RIGHTARC còn được gọi là các thao tác **reduce**, vì chúng vừa tạo arc vừa làm giảm số phần tử trên stack.

Điều kiện áp dụng các transition

Không phải lúc nào transition cũng hợp lệ.

- **LeftArc không được áp dụng nếu phần tử thứ hai là Root.**
 - Lý do: ROOT không thể có head đi vào nó.
- **LeftArc và RightArc cần ít nhất 2 phần tử trên stack.**
 - Vì cả hai thao tác đều tạo quan hệ giữa hai phần tử trên cùng.
- **Shift cần buffer chưa rỗng.**
 - Nếu buffer rỗng, không còn token nào để đưa vào stack.

Ý nghĩa

Các điều kiện này giúp parser không tạo cây sai về mặt cấu trúc, ví dụ không gắn head cho ROOT.

function DEPENDENCYPARSE(*words*) **returns** dependency tree

state $\leftarrow$ { [root], [*words*], [] } ; initial configuration

while *state* **not final**

t $\leftarrow$ ORACLE(*state*) ; choose a transition operator to apply

state $\leftarrow$ APPLY(*t*, *state*) ; apply it, creating a new state

return *state*

Hình: A generic transition-based dependency parser

Generic Transition-Based Dependency Parser

- 1 Khởi tạo trạng thái:

$$state \leftarrow ([\text{ROOT}], [w_1, \dots, w_n], \emptyset)$$

- 2 Trong khi chưa đạt trạng thái kết thúc:
 - 1 Oracle chọn transition t phù hợp với cấu hình hiện tại.
 - 2 Áp dụng t để sinh state mới.
- 3 Trả về tập quan hệ phụ thuộc R .

Độ phức tạp: parser đi qua câu một lượt từ trái sang phải, nên cách tiếp cận này có tính hiệu quả cao so với việc tìm kiếm toàn cục.

Tính chất của Arc-Standard Parsing

- Transition chỉ tạo quan hệ giữa các phần tử ở đỉnh stack.
- Khi một từ đã được gắn head, nó bị loại khỏi stack.
- Vì bị loại khỏi stack, từ đó không còn khả dụng cho các xử lý tiếp theo.

Hệ quả

Parser phải cẩn thận khi dùng `RIGHTARC` hoặc `LEFTARC`: nếu bỏ một từ khỏi stack quá sớm, parser có thể mất cơ hội gắn các dependent còn lại của từ đó.

Transition-Based Parsing

Bản chất của cách tiếp cận

Cách tiếp cận này là một thuật toán **tham lam** đơn giản. Ở mỗi bước, **oracle** chỉ đưa ra một lựa chọn duy nhất, và parser tiếp tục xử lý theo lựa chọn đó.

Đặc điểm

- Không xét các lựa chọn thay thế.
- Không sử dụng cơ chế quay lui.
- Cuối cùng chỉ trả về duy nhất một parse tree.

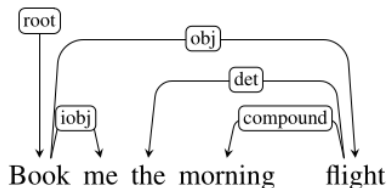
Hệ quả

Nếu oracle chọn sai ở một bước nào đó, lỗi có thể lan truyền sang các bước sau, vì parser không có cơ hội quay lại để sửa quyết định trước đó.

“Book me the morning flight”

Ví dụ 1:

Dependency tree mục tiêu có thể hiểu như sau:



Hình: Target Dependency tree cho Ví dụ 1

Ví dụ 1:

Minh họa cho state sau khi shift “Book” và “me” tại Step 2

$$S = [\text{ROOT}, \text{Book}, \text{me}], \quad B = [\text{the}, \text{morning}, \text{flight}], \quad R = \emptyset$$

Hành động đúng: dùng RIGHTARC để tạo quan hệ:

$$\text{Book} \rightarrow \text{me}$$

Sau đó *me* bị pop khỏi stack.

Ví dụ 1:

Step	Stack	Buffer	Action	Relation
0	[ROOT]	[<i>book, me, the, morning, flight</i>]	SHIFT	–
1	[ROOT, <i>book</i>]	[<i>me, the, morning, flight</i>]	SHIFT	–
2	[ROOT, <i>book, me</i>]	[<i>the, morning, flight</i>]	RIGHTARC	<i>book</i> →
3–5	[ROOT, <i>book, the, morning</i>]	[<i>flight</i>]	SHIFT	–
6	[ROOT, <i>book, the, morning, flight</i>]	[]	LEFTARC	<i>flight</i> →
7	[ROOT, <i>book, the, flight</i>]	[]	LEFTARC	<i>flight</i> →
8	[ROOT, <i>book, flight</i>]	[]	RIGHTARC	<i>book</i> →
9	[ROOT, <i>book</i>]	[]	RIGHTARC	ROOT →

Kết quả

Các relation được tích lũy tạo thành dependency tree hoàn chỉnh cho câu.

Greedy Parsing: Ưu điểm và hạn chế

Ưu điểm

- Nhanh, đơn giản, dễ cài đặt.
- Mỗi bước chỉ chọn một hành động.
- Phù hợp với mô hình học máy phân loại transition.

Hạn chế

- Một quyết định sai có thể kéo theo nhiều lỗi sau đó.
- Không quay lui để sửa lỗi.
- Có thể có nhiều chuỗi transition sinh cùng một cây, hoặc sinh các cây khác nhau.

Mở rộng : Labeled Tree

Để tạo cây có nhãn, ta mở rộng các transition bằng nhãn quan hệ.

Ví dụ

Thay vì chỉ có:

LEFTARC, RIGHTARC

ta có thể dùng:

LEFTARC(NSUBJ), RIGHTARC(OBJ), LEFTARC(DET),...

Hệ quả

Tập transition lớn hơn -> Nhiệm vụ của oracle khó hơn.

Oracle là gì?

Oracle là mô hình quyết định transition tiếp theo dựa trên configuration hiện tại.

Trong huấn luyện

Oracle được học bằng **supervised learning**. Dữ liệu huấn luyện cần có dạng:

(configuration, correct transition)

Vấn đề

Treebank thường cho ta cặp:

(*sentence*, *dependency tree*)

chứ không trực tiếp cho các cặp (*configuration*, *transition*).

Tạo dữ liệu huấn luyện từ treebank

- 1 Với mỗi câu trong treebank, lấy **reference parse**.
- 2 Chạy mô phỏng parser từ trạng thái ban đầu.
- 3 Ở mỗi configuration, dùng **training oracle** để chọn hành động đúng theo reference parse.
- 4 Ghi lại cặp:

$$(C_i, t_i)$$

- 5 Tập các cặp này trở thành dữ liệu huấn luyện cho classifier.

Ý nghĩa

Ta biến một dependency tree hoàn chỉnh thành một chuỗi quyết định nhỏ, phù hợp để huấn luyện mô hình phân loại.

Quy tắc chọn relation

Với một configuration hiện tại và reference parse, oracle chọn transition theo thứ tự ưu tiên:

- ❶ Chọn LEFTARC nếu hành động này tạo ra một quan hệ head-dependent đúng trong reference parse và configuration hiện tại.
- ❷ Nếu không, chọn RIGHTARC nếu:
 - quan hệ nó tạo ra đúng trong reference parse.
 - mọi dependent của từ đang xét ở đỉnh stack đã được gán relation.
- ❸ Nếu cả hai điều trên không thỏa, chọn SHIFT.

Trực giác

Oracle ưu tiên tạo arc khi chắc chắn đúng, nhưng không pop một từ khỏi stack nếu từ đó còn cần làm head cho từ khác.

Giả sử oracle có:

- Stack S , trong đó S_1 là đỉnh stack, S_2 là phần tử ngay dưới S_1 .
- R_c : tập relation đã tạo trong configuration hiện tại.
- R_p : tập relation trong reference parse.

Quy tắc

LEFTARC(r) : nếu $(S_1 \xrightarrow{r} S_2) \in R_p$

RIGHTARC(r) : nếu $(S_2 \xrightarrow{r} S_1) \in R_p$

và $\forall r', w : (S_1 \xrightarrow{r'} w) \in R_p \Rightarrow (S_1 \xrightarrow{r'} w) \in R_c$

SHIFT : otherwise

Vì sao RIGHTARC cần điều kiện phụ?

Trong arc-standard parsing, RIGHTARC sẽ pop từ ở đỉnh stack.

Nếu pop quá sớm

Từ bị pop sẽ không còn trên stack để làm head cho các dependent chưa xử lý.

Ví dụ:

Book the flight through Houston

Trong reference parse, có quan hệ:

Book $\rightarrow$ *flight*, *flight* $\rightarrow$ *Houston*

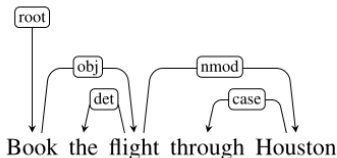
Nếu gán *Book* $\rightarrow$ *flight* quá sớm bằng RIGHTARC, từ *flight* bị pop, parser mất cơ hội gán *Houston* làm dependent của *flight*.

Ví dụ 2:

Với câu:

Book the flight through Houston

Một reference parse có thể biểu diễn bằng các relation:



Hình: Ví dụ 2

Mục tiêu của training oracle

Sinh ra chuỗi transition đúng để từ trạng thái ban đầu thu được đúng các relation trên.

Ví dụ 2: Sinh cặp huấn luyện

Step	Stack	Buffer	Action đúng
0	[ROOT]	[<i>book, the, flight, through, houston</i>]	SHIFT
1	[ROOT, <i>book</i>]	[<i>the, flight, through, houston</i>]	SHIFT
2	[ROOT, <i>book, the</i>]	[<i>flight, through, houston</i>]	SHIFT
3	[ROOT, <i>book, the, flight</i>]	[<i>through, houston</i>]	LEFTARC
4	[ROOT, <i>book, flight</i>]	[<i>through, houston</i>]	SHIFT
5	[ROOT, <i>book, flight, through</i>]	[<i>houston</i>]	SHIFT
6	[ROOT, <i>book, flight, through, houston</i>]	[]	LEFTARC
7	[ROOT, <i>book, flight, houston</i>]	[]	RIGHTARC
8	[ROOT, <i>book, flight</i>]	[]	RIGHTARC
9	[ROOT, <i>book</i>]	[]	RIGHTARC

Kết quả

Mỗi dòng là một training instance: configuration hiện tại kèm transition đúng.

Giải thích các bước thực hiện

- **Step 0–2:** dù $ROOT \rightarrow book$ là relation đúng, oracle chưa dùng RIGHTARC vì $book$ còn dependent là $flight$.
- **Step 3:** $flight \rightarrow the$ đúng trong reference parse, nên chọn LEFTARC để gắn the vào $flight$.
- **Step 4:** chưa dùng RIGHTARC cho $book \rightarrow flight$ vì $flight$ còn dependent là $Houston$.
- **Step 6:** $Houston \rightarrow through$ đúng, dùng LEFTARC.
- **Step 7–9:** khi các dependent đã hoàn tất, oracle có thể dùng RIGHTARC để lần lượt hoàn thiện cây.

Note

Training oracle không chỉ kiểm tra arc có đúng không, mà còn kiểm tra việc pop khỏi stack có làm mất thông tin cần thiết cho các bước sau hay không.

Cảm ơn!